Real-time Monitoring using AJAX and WebSockets

Darshan G. Puranik
Dept. of Computer and Info. Science
Indiana U.-Purdue U. Indianapolis
Indianapolis, IN USA
Email: dpuranik@cs.iupui.edu

Dennis C. Feiock
Dept. of Computer and Info. Science
Indiana U.-Purdue U. Indianapolis
Indianapolis, IN USA
Email: dfeiock@iupui.edu

James H. Hill
Dept. of Computer and Info. Science
Indiana U.-Purdue U. Indianapolis
Indianapolis, IN USA
Email: hillj@cs.iupui.edu

***Abstract*—Asynchronous JavaScript and XML (AJAX) is the primary method for enabling asynchronous communication over the Web. Although AJAX is providing warranted real-time capabilities to the Web, it requires unconventional programming methods at the expense of extensive resource usage. WebSockets, which is an emerging protocol, has the potential to address many challenges with implementing asynchronous communication over the Web. However, there has been no independent study that quantitatively compares AJAX and WebSockets.**

This paper therefore provides two contributions to integrating Web technologies in real-time systems. First, it provides an experience report for adding real-time monitoring support over the Web to the *Open-source Architecture of Software Instrumentation of Systems (OASIS)*, which is an open-source real-time instrumentation middleware for distributed real-time and embedded (DRE) systems. Secondly, its quantitatively compares using AJAX and WebSockets to stream collected instrumentation data over the Web. Results from our study show that a WebSockets server consumes 50% less network bandwidth than an AJAX server; a WebSockets client consumes memory at constant rate, not at an increasing rate; and WebSockets can send up to 215.44% more data samples when consuming the same amount network bandwidth as AJAX.

I. Introduction

Web 2.0 [1] technologies, such as Asynchronous JavaScript and XML (AJAX) [2], are revolutionizing how end-users interact with Web sites and Web applications. Instead of using many different pages and server callbacks to deliver content, Web 2.0 technologies enable Web sites to deliver content in real-time to Web clients while the end-user remains on the same web page. For example, it is possible to embed into an existing web page a real-time instant messaging widget that does not require the end-user to *refresh* the page, or visit different pages to send and/or receive messages.

Because of technologies like AJAX, web developers have open standards-based protocols built into the Web client that supports real-time monitoring capabilities via the Web. This is opposed to traditional methods that relied on embedded applets, and require developers to design, implement, and integrate proprietary networking protocol manually.

Within the AJAX realm, there are three primary patterns for asynchronous communication: *polling* [3], where the Web client sends a request at regular intervals and the Web server sends a response immediately then closes the connection; *long-polling* [4], where the Web client sends a request and the Web server keeps the connection open for an extended period of time; and *streaming* [5], where the Web server keeps the connection open indefinitely and stream responses to the Web client until the Web client terminates the connection.

Although AJAX is addressing many shortcomings of traditional Web development, *e.g.*, static web pages and language dependency, AJAX can be resource intensive in both memory usage and network bandwidth—especially when streaming content in real-time. This is because the AJAX Web server uses indefinite loops to stream content in real-time. Likewise, the semantics of how content is streamed and delivered causes new content to appended to the existing content until the existing connection is closed and reopened.

WebSockets [6], which is an emerging technology that integrates socket-like communication mechanisms into the Web, has the potential to address many challenges introduced by AJAX. For example, WebSockets does not inherently append new content to existing content as done with AJAX. Likewise, WebSockets provides raw socket capabilities to the Web. It is therefore possible to build—from the ground up—custom protocols using WebSockets that best suites the target application domain. This is opposed to force an existing protocol to operate in an unfit application domain.

WebSockets, however, is a relatively new technology and not supported by many browsers [7]. Because of this, it is not well-known how WebSockets compares with AJAX—the most prominent technology for real-time communication via the Web [8]—especially for real-time monitoring of DRE systems over the Web. Based on this understanding, the main contributions of this paper are as follows:

- It provides an experience report for enabling a real-time monitoring support of DRE systems via the Web using AJAX and WebSockets;
- It quantitatively compares using AJAX and WebSockets to enable real-time monitoring by measuring both client- and server-side performance metrics, such as network bandwidth, throughput, and memory usage; and
- It provides lessons learned for implementing real-time monitoring support via the Web using AJAX and WebSockets.

We performeda quantitative study in the context the *Open-Source Architecture for Software Instrumentation of Systems (OASIS)* [9], which is open-source real-time instrumentation middleware for distributed real-time and embedded (DRE) systems. OASIS enables real-time instrumentation of DRE systems without *a priori* knowledge of metric structure and

complexity. Likewise, instrumentation behavior can be modified at runtime to ensure minimal impact on software system performance. Finally, results from our study show that a WebSockets server consumes 50% less network bandwidth than an AJAX server; a WebSockets client consumes memory at constant rate, not at an increasing rate; and WebSockets can send up to 215.44% more data samples while consuming the same amount of network bandwidth when compared to AJAX.

Paper organization. The remainder of this paper is organized as follows: Section II provides a brief overview of OASIS; Section III explains how AJAX and WebSockets are integrated into OASIS; Section IV discusses the results of our comparative study; Section V compare our work with WebSockets and OASIS with other related works; and Section VI provides concluding remarks and lessons learned.

II. A Brief Overview of OASIS

OASIS is real-time instrumentation middleware for DRE systems that uses a metametics-driven design integrated with loosely-coupled data collection facilities. Figure 1 presents an high-level overview OASIS's architecture. As shown in this

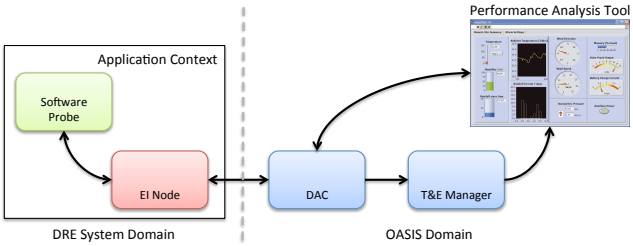


Fig. 1. A high-level overview of OASIS architecture and middleware.

figure, OASIS's architecture has the following key entities:

- **Software Probe.** The software probe is the entity that is responsible for collecting metrics from the DRE system under software instrumentation. Developers define software probes using the *probe definition language (PDL)*. The PDL is then used to generate base implementations for packing collected instrumentation data, and stubs for unpacking collected instrumentation data. System developers then have the option of inheriting the base implementation to define more domain-specific behavior for collecting instrumentation data, such as using system APIs to read the data points.

```
[uuid(ed970279-247d-42ca-aeaa-bef0239ca3b3)]
abstract probe MemoryProbe {
  uint64 physical_memory_avail, physical_memory_total;
  uint64 system_cache;
  uint64 commit_limit, commit_total;
  uint64 virtual_total, uint64 virtual_used;
};

[uuid(81DA0F4B-2712-4A7A-ABE4-F74C80A5C069)]
probe LinuxMemoryProbe : MemoryProbe {
  uint64 buffers, swap_cache;
  uint64 inactive, active;
  uint64 high_total, high_free, low_total, low_free;
  uint64 swap_total, swap_free;
  uint64 dirty, write_back;
  uint64 virtual_chunk;
};

[uuid(C78815F8-4A43-43BE-9E58-FE875E961B7D)]
probe WindowsMemoryProbe : MemoryProbe {
  uint64 page_file_total, page_file_avail;
  uint64 kernel_total, kernel_paged, kernel_nonpaged;
  uint64 page_size;
  uint64 commit_peak;
};
```

Listing 1. Source code snippet illustrating the memory software probe's PDL specification.

Listing 1 shows the PDL for a software probe that collects memory usage data from the host system. The base implementation for either the `LinuxMemoryProbe` or `WindowsMemoryProbe` is inherited to extract the data from `/proc` or the Windows Performance Counters on Linux and Windows hosts, respectively. Lastly, software probes can be client-driven or active objects.

- **Embedded Instrumentation Node.** The Embedded Instrumentation (EI) Node bridges locality constrained abstractions with networking abstractions. When the EI Node receives collected instrumentation data as a data packet, it prepends its information (*e.g.*, UUID, packet number, timestamp, and hostname) to the data packet, and sends it over the network.
  The EI Node is not bound to a specific network communication protocol, or technology. For example, the EI Node can use an implementation of CORBA [10] (*e.g.*, The ACE ORB (TAO) [11]) or Data Distribution Services (DDS) [12] (*e.g.*, RTI-DDS (www.rti.com) and OpenSplice (www.prismtech.com/opensplice)) to send the fully packaged EI Node instrumentation data. This design approach allows DRE system developers to select the most appropriate networking middleware for their domain without impacting how OASIS packages instrumentation data. Lastly, there is one EI Node per application context (*i.e.*, an execution block, such as a for loop or conditional, an object/class, a component, or single application).
- **Data Acquisition and Controller.** The Data Acquisition and Controller (DAC) is responsible for receiving packaged data from an EI Node and controlling access to it. The DAC also manages *data handlers*, which are objects that act upon instrumentation data received from an EI Node. For example, an archive data handler stores collected metrics in a relational database, and a real-time publisher data handler allows clients to register for instrumentation data and receive it in real-time. This design approach allows OASIS to abstract away the data collection facilities from its data handling facilities, and places the data handling facilities outside of the DRE system's execution domain.
- **Test and Execution Manager.** The Test and Execution (TnE) Manager is a naming service for DACs. It is therefore the main entry point into OASIS for clients that want to access collected instrumentation data.
- **Performance Analysis Tools.** The performance analysis tools are clients that use instrumentation data collected by OASIS. Examples of performance analysis tools include,

but is not limited to: real-time event processing engines and dashboards. Lastly, performance analysis tools can send signals/commands to software probes that alter its behavior at runtime. This design enables system developers, system testers, and performance analysis tools to control the effects of software instrumentation at runtime and minimize OASIS's overhead.

With the recent advances in Web technologies, such as AJAX and WebSockets, it is now possible to leverage the Web to monitor DRE systems in real-time. It, however, is unknown what impact such technologies have on this domain. Moreover, WebSockets is a fairly new technology when compared to AJAX. It is therefore unknown which technology is better for this domain. The remainder of this paper therefore discusses how AJAX and WebSockets are integrated into OASIS, and compares the performance of the two technologies.

III. Integrating WebSockets and AJAX into OASIS

The previous section provided an overview of OASIS. As discussed in that section, the data handler is an integral part of OASIS. This is because the data handler processes instrumentation data received by the DAC outside of the DRE system undergoing software instrumentation. Because we want to integrate both AJAX and WebSockets into OASIS—as explained in Section II—and compare its performance, the data handler is the best location to perform this integration because it ensures minimal impact on the DRE system's performance. The remainder of this section therefore discusses how we integrated an AJAX and WebSockets data handler in OASIS with the goal of comparing their performance and applicability in real-time instrumentation and monitoring of DRE systems.

A. Integrating AJAX into OASIS

Figure 2 provides an overview of how AJAX is integrated into OASIS. There are various design and implementation methods for AJAX-enabled web application as introduced in Section I. We implemented the streaming pattern [5] for AJAX because it most closely matches our design needs. As shown in this figure, AJAX is integrated as a DAC data handler. When the DAC receives instrumentation data, it is forwarded to the AJAX data handler. The AJAX data handler then unpacks the instrumentation data, and writes it to a local file on disk in JavaScript Object Notation (JSON) [13] format. This is similar to writing the instrumentation data to a database.[1]

We then implemented a simple PHP (www.php.net) script that reads the values from the text file updated by AJAX data handler. The script is written in such a way that it executes an infinite loop while checking for updates to the text file. If an update is detected, then the script reads the values from the local file and streams it to the Web client. Because of the streaming pattern, the Web server keeps the HTTP connection open indefinitely. Lastly, the script is hosted in an Apache Web server.

[1]We are aware this design approach will produce biased results that do not favor AJAX. Based on our investigations, however, we learned this is the most common approach used in the real-world for creating an AJAX-enabled web application. More optimized approaches include creating an in-memory buffer that replaces the flat-file (or database).

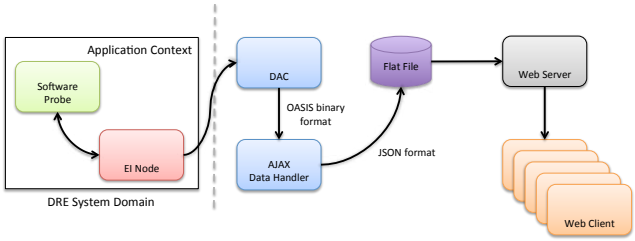


Fig. 2. High-level overview of integrating of AJAX into OASIS.

For this integrated version, we designed a simple Web application that uses the XMLHttpRequest object to open a connection to the AJAX data handler and receive instrumentation data in real-time. When the Web application receives a new JSON message it locates the last data sample received, and updates an HTML table with it.

B. Integrating WebSockets into OASIS

Figure 3 provides an overview of how WebSockets is integrated into OASIS. As shown in this figure, WebSockets is integrated into OASIS as a data handler. Unlike the AJAX integration, there is no intermediate step between the WebSockets data handler and the Web application. Instead, as the WebSockets data handler is forwarded instrumentation data by the DAC, it sends the instrumentation data to the Web application in the same binary format.

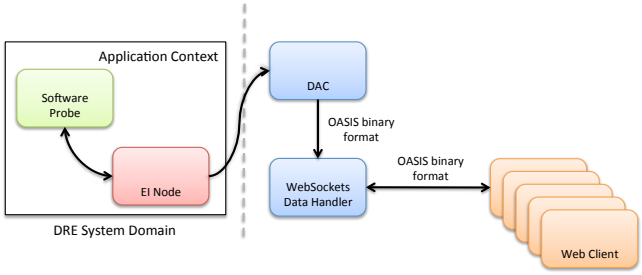


Fig. 3. High-level overview of integrating WebSockets into OASIS.

In comparison to the AJAX data handler (explained in Section III-A), the WebSockets data handler's design and implementation is more complex as shown in Figure 4. We implemented the WebSockets data handler using the Adaptive Communication Environment (ACE) [14], which is a widely-used C++ framework for writing portable networked applications, and used in DRE systems [15]. We also used ACE to implement the WebSockets data handler because its Acceptor/Connector framework simplified many networking challenges such as reading/writing data in the correct byte order; reading/writing frames, which is an integral part of the WebSockets protocol; and managing connections between multiple Web applications.

As shown in Figure 4, the WebSockets data handler is composed of the following key abstractions that are designed

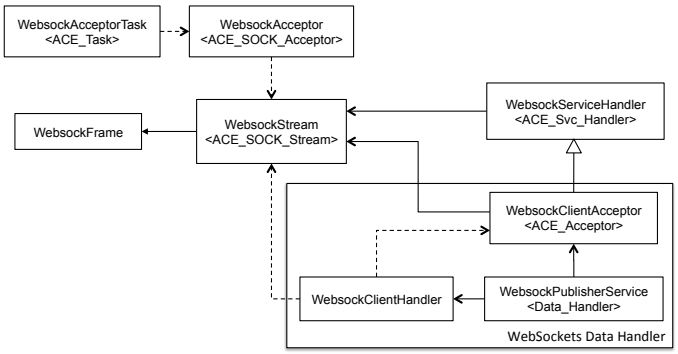


Fig. 4. Architectural diagram of the WebSockets data handler implemented using ACE.

to be used by any WebSockets client/server:

- **WebsockAcceptorTask.** The WebsocketAcceptorTask is an active object that extends the ACE_Task class in ACE. This object executes N number of threads that run an event loop of an ACE_Reactor object. The ACE_Reactor object is an object that dispatches events, such as input handle events and timeout events, to the task executing the reactor's event loop. This simplifies handling input events from the Web application.
- **WebsockAcceptor.** The WebsockAcceptor class extends the ACE_SOCK_Acceptor class in ACE. The purpose of the WebsockAcceptor object is to listen to incoming connections from the Web application on a specified port. When the WebsockAcceptor is notified of an incoming connection, it accepts the connection and creates a stream for sending/receiving data to/from the Web application. This object then authenticates itself with the Web application as per the WebSockets specification. If the authentication succeeds, the WebSockets data handler can begin communicating with the Web application. If the authentication fails, then the connection is closed.
- **WebsockStream.** The WebsockStream class extends the ACE_SOCK_Stream class in ACE. This object is responsible for sending/receiving data to/from the Web application. The WebsockStream also abstracts away the framing complexity of the WebSockets protocol with simple send/receive methods that take a data buffer. The WebsockStream then uses a special data structures to pack/unpack the data accordingly to the WebSockets protocol.
- **WebsockServiceHandler.** The WebsockServiceHandler extends the ACE_Svc_Handler class. This class is responsible for notifying WebsockStream objects when data from a Web application is ready for reading.
- **WebsockFrame.** The WebsockFrame class is a helper class that builds frames according to WebSockets protocol. It is primarily used by WebsockStream objects.

In addition to the generic abstractions discussed above, the following abstractions are specific to the WebSockets data handler:

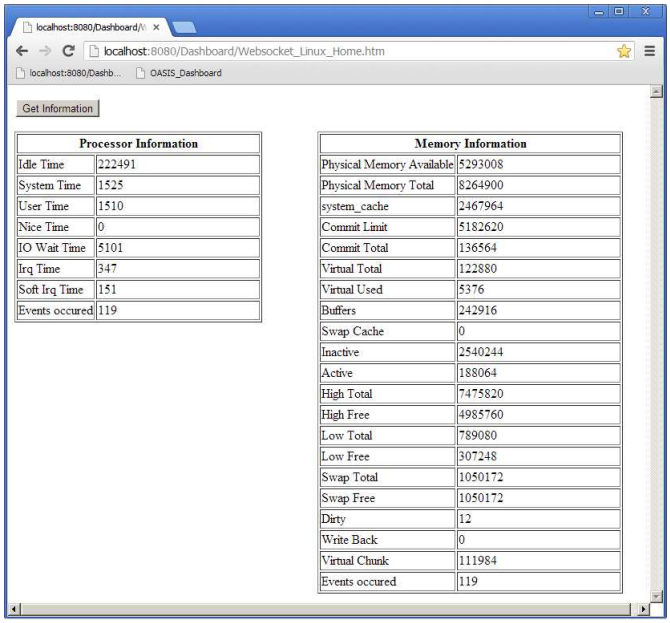
- **WebsockClientHandler.** The WebsockClientHandler extends the ACE_Service_Handler. This object adds an extra level of indirection to ACE's reactor design, but adds more flexibility when sending instrumentation data to the Web application.
- **WebsockClientAcceptor.** The WebsockClientAcceptor class extends the ACE_Acceptor class in ACE. This object is a factory for WebsockClientHandler objects. When it creates a new WebsockClientHandler, the WebsockClientAcceptor registers it with the system's reactor. This object is also responsible for managing the subscription status for instrumentation data for connected Web applications.
- **WebsockPublisherService.** The WebsockPublisherService implements the DAC's data handler interface. This object is where OASIS integrates with WebSockets. When the DAC receives instrumentation data, it is forwarded to this object. The WebsockPublisherService then forwards the instrumentation data to the WebsockClientHandler, which is responsible for distributing the data accordingly.

When WebSockets sends instrumentation data to the Web application, it is in binary format and packaged according to OASIS's packaging specification. Therefore, we had to implement JavaScript classes that converted the OASIS binary data to standard types in JavaScript. This also included resolving byte order issues, if they were applicable. Once the Web application converts the received binary data to its equivalent JavaScript types, the Web application updates an HTML table with the latest values from the data sample—similar to the Web application used with the AJAX data handler.

IV. Comparing the Performance of AJAX and WebSockets

This section discusses experimental results for integrating AJAX and WebSockets into OASIS to enable real-time monitoring of DRE systems as discussed in Section III. The experimental results discussed in this section focus on the following three performance properties:

- **Web application memory consumption.** This property focuses on how much memory the Web application consumes while receiving instrumentation data from the DAC using either AJAX or WebSockets. We selected this performance property because the Web application is an integral part of real-time monitoring that must run for extended periods of time. The Web application therefore should run as efficiently as possible on general-purpose computers (*e.g.*laptops, mobile phones, and tablets). This will allow the end-user to take advantage of real-time monitoring from any place that supports an Internet connection.
- **Network bandwidth consumption.** This property is concerned with evaluating how much network bandwidth the AJAX and WebSockets implementation use. We selected this performance property for two reasons. The first reason is because of economics. Network bandwidth is



Processor Information	
Idle Time	222491
System Time	1525
User Time	1510
Nice Time	0
IO Wait Time	5101
Irq Time	347
Soft Irq Time	151
Events occured	119

Memory Information	
Physical Memory Available	5293008
Physical Memory Total	8264900
system_cache	2467964
Commit Limit	5182620
Commit Total	136564
Virtual Total	122880
Virtual Used	5376
Buffers	242916
Swap Cache	0
Inactive	2540244
Active	188064
High Total	7475820
High Free	4985760
Low Total	789080
Low Free	307248
Swap Total	1050172
Swap Free	1050172
Dirty	12
Write Back	0
Virtual Chunk	111984
Events occured	119

Fig. 5. Screenshot of the basic Web application used to display instrumentation data received in real-time using either AJAX or WebSockets.

costly, especially with mobile phone carriers now placing restrictions on network bandwidth consumption [16]. This implies that network bandwidth usage should be kept as minimal as possible to ensure that real-time monitoring is affordable. The second reason is because real-time monitoring is inherently data-intensive. This implies that network congestion can easily become a problem, and delay receipt of collected instrumentation data.

- **Data throughput and data lag.** These properties are concerned with evaluating how much data AJAX and WebSockets can handle when integrated into OASIS. We selected these performance properties because it provides insight on their capacity and scalability.

We developed two sample Web applications for our experiments. The first application used AJAX to send instrumentation data in real-time from the DAC to the Web application (see Section III-A). The second application used WebSockets to send instrumentation data in real-time from the DAC to the Web application (see Section III-B). We used the System Probe Daemon tool, which is a tool provided with OASIS, to collect processor and memory information from each host in the experiment. Lastly, the Web applications were executed in Google Chrome 19 and displayed received instrumentation data in table format. Figure 5 shows a screen shot of the Web application without any instrumentation data.

All experiments were conducted in the System Integration (SI) Lab (www.emulab.cs.iupui.edu) at IUPUI, which is powered by Emulab [17] software. Each experimental node in the SI Lab is a Dell PowerEdge R415, AMD Opteron 4130 processor with 8GB of memory executing 32-bit Fedora Core 15 (32 bit). *Boss* is Dell PowerEdge R415, AMD Opteron 4130 processor, 8GB of memory executing 32-bit FreeBSD 7.3.

For each experiment execution, the System Probe Daemon tool, DAC, and TnE Manager were deployed on their own experimental node. We only used one DAC in the experiments because we are not focusing on scaling the OASIS architecture with respect to streaming instrumentation data to the Web application. Finally, the Google Chrome web browser (*i.e.*, the performance analysis tools) was deployed on a Dell XPS 15z laptop with Intel Core i5 processor and 6 GB of memory executing 64-bit Windows 7 Ultimate. The laptop resided outside of the SI Lab, and the instrumentation data was sent over the public Internet using a WiFi connection. The remainder of this section discusses the results of our experiments evaluating the three performance properties discussed above.

A. Experiment 1: Web Application Memory Consumption

The goal of this experiment is to compare memory consumption on the client-side (*i.e.*, measure how much memory the web browser is using) when using AJAX and WebSockets to monitor collected instrumentation data in real-time.

1) Experiment Design & Setup: Using the general experimental setup explained at the beginning of this section, we configured the System Probe Daemon tool to collect instrumentation data at 1 Hz. We selected 1 Hz because it allowed us to stream collected instrumentation data in real-time using both AJAX and WebSockets under similar operating conditions. When designing this experiment, we learned that if we collect instrumentation data at too high of a rate, then the AJAX data handler publishes data at a lesser rate than the WebSockets data handler. This is because the AJAX design has a "middle-man" (*i.e.*, the flat file) that enables streaming, and the "middle-man" introduces a delay that is not present in the WebSockets experiment. Finally, we executed the tests for 15 minutes and collected memory consumption metrics for the Web application using Windows command-line tool named *Tasklist* at 30 second intervals.

2) Experiment Results: Figure 6 shows the memory consumption results for AJAX and WebSockets when integrated into OASIS. As also shown in Figure 6, memory consumption for the Web application that uses AJAX increases over time, and Web application memory consumption for the WebSockets implementation remains relatively constant. This is because AJAX implements the streaming pattern by appending new messages to previously received messages. This causes the response to increase in size over time and causes the Web client to consume more memory over time.

In case of WebSockets, each message is transferred in its own frame, or set of frames. The Web application that uses WebSockets therefore consumes an amount of memory that is consistent with the amount of memory that represents only the latest data sample. This analysis, however, disregards any memory consumed by the Web application in regards to storing and interacting with the received data. Finally, because of how we had to design the experiment to ensure fair comparison between AJAX and WebSockets, we received our first insight that AJAX Web servers cannot stream content as fast as

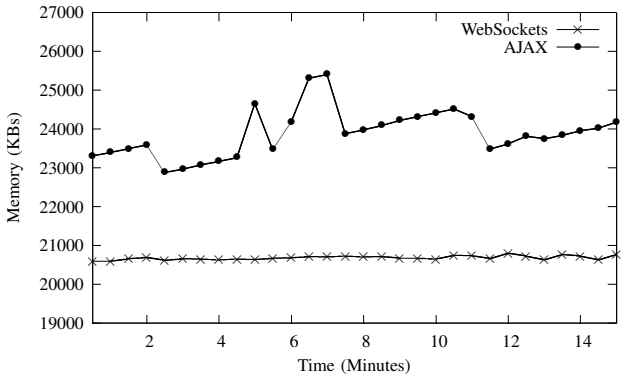


Fig. 6. Results comparing Web application memory consumption between AJAX and WebSockets when integrated into OASIS.

WebSockets Web servers. This is illustrated in more detail in Section IV-C.

B. Experiment 2: Network Bandwidth Consumption Test

The goal of this experiment is to compare network bandwidth consumption between the AJAX and WebSockets implementation when integrated into OASIS to enable real-time monitoring of DRE systems.

1) Experiment Design & Setup: Using the general experimental setup explained at the beginning of this section, we configured the software probe to flush a fixed number of data points. This is because we wanted to ensure that both AJAX and WebSockets had the same amount of workload. If we allowed the software probe to run for a fixed amount of time, then the comparison would be unfair. This is because we learned from the previous experiment that WebSockets can operate at a much higher rate than AJAX, and the comparison of network bandwidth between both implementations would not be under the same operating conditions.

We used WireShark (www.wireshark.org), which is an open-source tool for monitoring packets on a network, in this experiment. More specifically, we used WireShark to monitor only the packets sent between the DAC and the Web application by measuring the number of bytes associated with each packet. We added number of bytes associated with each packet to come up with total number of bytes transferred over network for one experiment. Finally, we executed the experiment 10 different times using 10 different number of fixed software probe flushes.

2) Experiment Results: Figure 7 shows the network bandwidth consumption results for AJAX and WebSockets when integrated into OASIS. As shown in this figure, the Web application that uses AJAX always consumes more network bandwidth than the Web application that uses WebSockets. There are two main reasons behind this observation. First, we learned that AJAX requires at least 256 bytes of data per message. We therefore had to add 256 bytes of whitespace to every message if its original size was under 256 bytes. This requirement causes unnecessary bandwidth usage. Second, we observed that AJAX's header size is unpredictable, but it is always significantly greater than its equivalent in WebSockets.

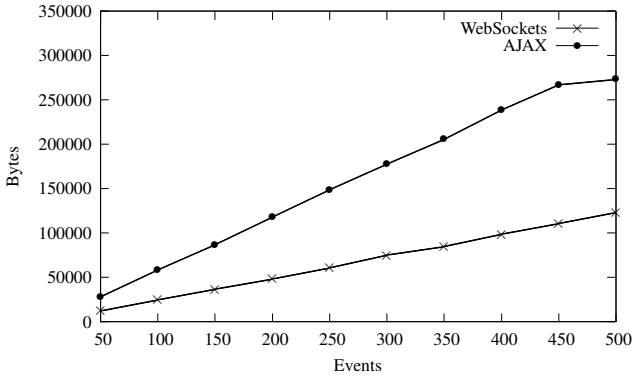


Fig. 7. Results comparing network bandwidth consumption between AJAX and WebSockets when integrated into OASIS.

C. Experiment 3: Data Throughput and Data Lag Test

The goal of this experiment is to compare data lag between WebSockets and AJAX when integrated into OASIS and the System Probe Daemon tool is collecting instrumentation data at its maximum rate.

1) Experiment Design & Setup: Using the general experimental setup explained at the beginning of this section, we configured the System Probe Daemon tool to collect and send instrumentation data to the DAC as fast as possible, which was then sent to the Web application. We then counted the number data samples sent to the DAC and the number of data samples received by the Web client. We designed the experiment in this way because we wanted to compare real-time performance of WebSockets and AJAX under extreme conditions. This experiment also allows us to establish maximum throughput for both technologies. Finally, each test was executed for 1 minute.

2) Experiment Results: Table I and Table II presents results that measure both throughput and data lag, which we define as the percentage of events sent by the server that have not been received by the client over a period of time, for AJAX and WebSockets when integrated into OASIS. As shown in the Table I, the WebSockets data handler sent a total of 161638 data samples (*i.e.*, processor and memory software probe data sample), and the Web application received all the data samples within 1 minute time frame. This means that the WebSockets implementation has no data lag, but this does not mean the WebSockets implementation did not experience latencies. The latencies were low enough for our experiments that each data sample sent was received within the allotted time period.

The AJAX implementation, however, had different results. As shown in Table II, the AJAX data handler sent a combined 161323 data samples (*i.e.*, processor and memory software probe data sample), but the Web application only received 1321 of the sent data samples. For this experiment, the AJAX implementation has a 98% data lag for 1 minute time period.

	Avg. Packet Size (Bytes)	Samples Sent	Samples Received	Data Lag
Processor Probe:	140	94,294	94,294	0%
Memory Probe:	236	67,344	67,344	0%
Total:	376	161,638	161,638	0%

TABLE I
THROUGHPUT AND DATA LAG RESULTS FOR WEBSOCKETS WHEN INTEGRATED INTO OASIS.

	Avg. Packet Size (Bytes)	Samples Sent	Samples Received	Data Lag
Processor Probe:	453	93,312	462	99.5%
Memory Probe:	733	68,011	859	98.73%
Total:	1,186	161,323	1,321	99.18%

TABLE II
THROUGHPUT AND DATA LAG RESULTS FOR AJAX WHEN INTEGRATED INTO OASIS.

Based on our investigations, we believe the data lag in the AJAX results is caused by two factors. The first factor is related to significant networking overhead. This is because the AJAX data handler receives data samples as packaged binary data and converts it to text-based data samples before sending it to the Web application. This conversion process negatively impacts its performance.

The second factor is related to how the data is received on the client-side. Although AJAX is sending only the latest data sample, it is appended to previously received samples. This means that the Web application must sift through all previously received data samples to locate the latest data sample, which will have at least a linear degradation on performance. One solution to addressing this problem is to open and close the connection continuously (*i.e.*, use a polling approach). This approach, however, will add more stress to the client and server, and reduces the overall throughput of data samples.

Lastly, Table I shows that WebSockets sends more data samples than AJAX for the same period of time. This can raise concerns that WebSockets can potentially use more bandwidth within a given time period when compared to AJAX. From Table I and Table II, we can calculate that the total amount of data sent in the AJAX experiment was 1,566,706 bytes for 1,321 data samples. We can then use this amount to determine what is the equivalent number of data samples sent using WebSockets that will produce the same quantity of data sent, which is 4,167 data samples. Using this number, we can calculate that for the same amount of data, WebSockets sends up to 215.44% more data samples than the AJAX implementation. This also means that we can reduce the sending rate of WebSockets, and still send the same amount of data or more while consuming less networking bandwidth.

Based on these experimental results, we can conclude that WebSockets is a better Web technology for real-time monitoring via the Web because of its high throughput and low bandwidth requirements.

V. RELATED WORKS

Dakshita [18] is a web-based real-time web-based monitoring condition monitoring system for power transformers. Dakshita collects data from hardware sensors and stores collected data in an Oracle database. It then uses AJAX to stream content to a Web application. As per our work integrating both AJAX and WebSockets into OASIS, we have learned that their approach is pseudo real-time. This is because storing and retrieving data from the database is time-consuming, and increases the chance of retrieving stale, or out-of-date, data. Our performance testing also shows that WebSockets is a potential solution to resolving such issues that may arise.

Cara [19] is a web-based real-time remote monitoring system for pervasive healthcare that uses Flex (www.adobe.com/products/flex.html) and FluorineFx.Net (www.fluorinefx.com). Within Cara, sensors collects data and transmit it a gateway using Bluetooth or Wi-Fi. The gateway then streams the data to the Cara server using Adobe Flash. End-users can then view the data in real-time by logging in to the Cara server. Experiments were conducted to measure Cara's networking latency on different networks. The experiments revealed that Cara experiences high network latencies, which is attributed to high network bandwidth usage. Our experiments also show that AJAX, which is similar to Flex, has high network bandwidth usage. Lastly, WebSockets supports data fragmentations (*i.e.*, data can be divided into multiple frames and transferred independently), which can be useful for Cara's video streaming feature.

StreamWeb [20] is a real-time web monitoring system with stream computing application domain that is developed atop of a stream computing system called System S [21]–[23] developed by IBM Research. Under the hood, StreamWeb uses AJAX to stream content to the Web application in real-time. StreamWeb, however, does not keep the AJAX connection open between multiple request for content in real-time. We believe this is one approach to reduce network bandwidth and memory consumption experience with AJAX, but it hinders stream content in real-time at high rates. We therefore believe that WebSockets can be used to address this design challenge, and enable updates at higher rates since the connection between the Web application and the server remains open.

Lastly, Websocket.org provides interesting results that compare the performance of WebSockets and Comet [24]. Comet is web technology that uses long-polling technique to achieve real-time behavior. According to the results, Websocket.org shows that WebSockets has better throughput and less net-

work latency when compared to Comet. Our results not only complement and extend their experimental results, it increases support for using WebSockets (an emerging Web technology) to enable real-time behavior via the Web when compared to AJAX, and similar Web technologies.

VI. Concluding Remarks

The advent of Web 2.0 technologies, such as AJAX and WebSockets, is allowing the Web to be applied to application domains that had to use either *ad hoc* or custom solutions to realize the same capability. One such application domain is real-time instrumentation and monitoring of DRE systems. In this paper, we compared the performance of using AJAX and WebSockets to support real-time instrumentation and monitoring of DRE systems. Based on our results, we can conclude that WebSockets is a better fit for this domain because it provides higher throughput and better network performance when compared to AJAX. Based on our experience in comparing the performance of AJAX and WebSockets, the following is a list of lessons learned and future research directions:

- **The need for improved client-side programming languages.** During our experiments, we observed that JavaScript was causing performance degradation in both the AJAX and WebSockets Web application. In today's society, general-purpose computers and devices, such as desktops, laptops, and mobile devices, have a significant amount of processing power and memory availability. It is therefore possible to shift some of the heavy lifting traditionally done by the server, such as chart generation on-the-fly, to the client (*i.e.* the general-purpose computer and device). Unfortunately, we learned that JavaScript inherently makes it *hard* to use advance programming techniques to design and implement solutions that were originally designed and implemented for a server. We therefore believe that in order to truly migrate workloads from the server to the client, we need to improve client-side programming (or scripting) languages.
- **The need for improved client-size charting and graphing libraries.** During our experiments, we observed that the WebSockets implementation could transfer more than 3000 events/second. Originally, we implemented the WebSockets Web application to visualize data using *RGraph* (www.rgraph.net), which is an open-source charting library written in JavaScript that uses HTML 5 features, such as the Canvas element, to dynamically create charts on the client-side. When we executed the WebSockets experiments, however, RGraph could not handle more than 100 events/second. Real-time monitoring over the Web need efficient graphing libraries to take full advantage of WebSockets performance, or any other Web technology that enable real-time event notification at high rates. Future research therefore includes improving the performance of client-side graphing libraries so it can operate in domains that have high throughout (or update) rates.
- **The advantage of AJAX streaming pattern**. During the data throughput and data lag experiment, we concluded that performance was degrading because AJAX was appending new data samples to previously received ones. This was causing the Web application to parse the entire response just to locate the latest data sample at the end. Although this can be viewed as a shortcoming for AJAX, it can be viewed as an advantage in application domains where historical content is displayed along with new content. When using WebSockets, the Web application has to manually implement this feature.

OASIS, the AJAX data handler, and WebSockets data handler discussed in this paper are freely available in open-source format from the following location: oasis.cs.iupui.edu. Lastly, we are in the process of merging our generic WebSockets abstractions into the ACE code base so it is available to the entire ACE community.

References

[1] OReilly, T.: What is Web 2.0: Design Patterns and Business Models for the Next Generation of Software. Communications & Strategies **1** (2007) 17–38

[2] Holdener III, A.: Ajax: the definitive guide. O'Reilly (2008)

[3] Takagi, H.: Analysis of polling systems. MIT press (1986)

[4] Russell, A.: Comet: Low latency data for browsers. alex. dojotoolkit. org (2006)

[5] Fecheyr-Lippens, A.: A Review of HTTP Live Streaming. Technical report, Technical report, http://andrewsblog. org/a review of http live streaming. pdf (2010)

[6] Internet Engineering Task Force (IETF): The WebSocket Protocol. Request for Comments: 6455 edn. (December 2012)

[7] Wikipedia: WebSocket. http://en.wikipedia.org/wiki/WebSocket (2012)

[8] BuiltWith Technology Lookup: AJAX Libraries API Usage Statistics. http://trends.builtwith.com/cdn/AJAX-Libraries-API (2012)

[9] Hill, J.H., Sutherland, H., Staudinger, P., Silveria, T., Schmidt, D.C., Slaby, J.M., Visnevski, N.: OASIS: An Architecture for Dynamic Instrumentation of Enterprise Distributed Real-time and Embedded Systems. International Journal of Computer Systems Science and Engineering, Special Issue: Real-time Systems (April 2011)

[10] Object Management Group: The Common Object Request Broker: Architecture and Specification Version 3.1, Part 1: CORBA Interfaces. OMG Document formal/2008-01-04 edn. (January 2008)

[11] Schmidt, D.C., Natarajan, B., Gokhale, A., Wang, N., Gill, C.: TAO: A Pattern-Oriented Object Request Broker for Distributed Real-time and Embedded Systems. IEEE Distributed Systems Online **3**(2) (February 2002)

[12] Object Management Group: Data Distribution Service for Real-time Systems Specification. 1.2 edn. (January 2007)

[13] Crockford, D.: JSON: Javascript object notation (2006)

[14] Huston, S.D., Johnson, J.C.E., Syyid, U.: The ACE Programmer's Guide. Addison-Wesley, Boston (2002)

[15] Group, D.: ACE Success Stories. www.cs.wustl.edu/~schmidt/ACE-users.html

[16] David Goldman: Sorry, America: Your Wireless Airwaves are Full. http://money.cnn.com/2012/02/21/technology/spectrum_crunch/index.htm (February 2012)

[17] Lepreau, J.: The Utah Emulab Network Testbed

[18] Wagle, A., Lobo, A., Kumar, A.S., Patil, S., Venkatasami, A.: Real time Web Based Condition Monitoring System for Power Transformers-Case Study. In: International Conference on Condition Monitoring and Diagnosis, IEEE (2008) 1307–1309

[19] Yuan, B., Herbert, J.: Web-based real-time remote monitoring for pervasive healthcare. In: Pervasive Computing and Communications Workshops (PERCOM Workshops), 2011 IEEE International Conference on, IEEE (2011) 625–629

[20] Suzumura, T., Oiki, T.: StreamWeb: Real-Time Web Monitoring with Stream Computing. In: Web Services (ICWS), 2011 IEEE International Conference on, IEEE (2011) 620–627

[21] Gedik, B., Andrade, H., Wu, K., Yu, P., Doo, M.: SPADE: the system s declarative stream processing engine. In: Proceedings of the 2008 ACM SIGMOD international conference on Management of data, ACM (2008) 1123–1134

[22] Jain, N., Amini, L., Andrade, H., King, R., Park, Y., Selo, P., Venkatramani, C.: Design, implementation, and evaluation of the linear road bnchmark on the stream processing core. In: Proceedings of the 2006 ACM SIGMOD international conference on Management of data, ACM (2006) 431–442

[23] Wolf, J., Bansal, N., Hildrum, K., Parekh, S., Rajan, D., Wagle, R., Wu, K., Fleischer, L.: SODA: An optimizing scheduler for large-scale stream-based distributed computer systems. Middleware 2008 (2008) 306–325

[24] Gravelle, R.: Comet Programming: Using Ajax to Simulate Server Push. Webreference, WebMediaBrands Inc., http://www. webreference. com/programming/javascript/rg28 (2010)